



Sealed group predictions, proved on your phone.

Slip is an iOS app for calling things with friends. You commit to a pick, it stays hidden, and later you prove your opening matches the seal you made — without anyone having to take your word for it.

THE PROBLEM

Every pick'em app promises nobody peeked.

The promise rests on a server you cannot inspect, run by people you do not know. When the group chat argues about who actually called it, the evidence is a screenshot and a claim.

A prediction is only interesting if it was *fixed* before the outcome. That is a cryptographic property, and today it is enforced by trust.

THE LOOP

Commit. Seal. Open. Prove.

Pick

Choose a side. The choice is witness data; its salt is derived in-circuit from the device secret, never supplied by the host.

Seal

Hold to seal. The phone generates the zero-knowledge proof itself. In this Wave 1 build nothing is submitted — proof and commitment stay on the device.

Open

At the deadline the pick opens and is checked against its commitment by the circuit.

Call

The outcome is settled separately. Opening establishes what was sealed, not what happened.

A reveal that does not match its commitment is rejected by the contract, not smoothed over by the client.

WHAT MAKES IT DIFFERENT

The seal proof is generated on the phone, by the phone.

MidnightKit is our Swift package wrapping `midnight-zk` and `midnight-ledger` as a Rust static library for `aarch64-apple-ios`, running the Compact runtime under JavaScriptCore.

- Removing the proof server is not an optimisation. **A proof server is exactly the party that would otherwise have to be trusted.**
- The witness — `{choice, salt}` — is never logged, never persisted unencrypted, never transmitted. **Only the commitment and the proof travel.**
- One Compact contract, six circuits: `enrollMember` · `createSlip` · `sealPick` · `reveal` · `settle` · `dispute`

EVIDENCE

Five circuits, proved on the device, in one run.

iPhone 17 Pro simulator, from the run recorded in the demo video. The README's physical-device and older-runtime tables report their own separate numbers.

CIRCUIT	PREPARE	KEY LOAD	PROOF	SIZE
Add the local player	39.0 ms	15.0 ms	520 ms	4,480 B
Create the round	22.0 ms	14.0 ms	504 ms	4,480 B
Seal the pick	23.5 ms	26.0 ms	969 ms	4,480 B
Open the pick	35.9 ms	30.0 ms	950 ms	4,480 B
Call the outcome	34.8 ms	15.0 ms	644 ms	4,480 B

The app shows these itself, under the line “Measured locally. These proofs have

QUALITY

Verified, not asserted.

140

tests · 22 suites

Including 120 reference renders and a privacy test asserting the shared invite payload carries no witness material.

5,238 B

proved tx · 1,256 ms

Native prover → authenticated relay → local devnet, reaching SucceedEntirely with the indexed commitment matching exactly. Host-produced; not physical-iPhone provenance.

0.31.1

compiler · ledger 8

Pinned to the stack Midnight's networks actually run. A freshness gate fails the build on drift.

What this does not claim.

- **Connected crews are not shipped.** `enrollMember` is gated by `assertSteward()`, so joining is not self-service. Tapping an invite imports public round metadata; it does not establish on-chain membership. The app says so on screen.
- **The deployment is a local devnet**, not a public testnet.
- **The proof establishes that an opening matches its seal.** It does not establish that nobody saw an unlocked phone.

These are in the README in the same words. A privacy product that overstates itself has already failed at the thing it sells.

NEXT

Wave 2

- **Steward-side enrolment through the relay**, so a crew can be assembled before a round opens — the honest version of “joining”.
- **Public testnet deployment**, moving off the local devnet.
- **TestFlight**, putting the app in real testers’ hands.
- **The two-device round**: two phones, one round, both seal, both open, standings agree.

Apache-2.0

midnightntwrk

compact